INDEX

Practical No.	Aim	Date	Sign
1	To perform fundamental data handling and processing operations using Python, including: 1. Parsing and extracting data from text, CSV, HTML, XML and JSON files 2. Performing read and write operations on binary files 3. Searching, splitting and replacing strings using regular expressions 4. Creating and managing a relational database using SQL CRUD operations		
2	To perform the ETL process in Power BI by extracting, transforming, and loading data to prepare it for analysis.		
3	To implement and demonstrate single and multiple document insertion and retrieval operations in MongoDB.		
4	To perform noise elimination, feature selection, and exploratory data analysis (EDA) using Python libraries.		
5	To extract user data from a REST API and a flat CSV file, transform and merge the data using Python.		
6	To design and configure a scheduled Apache Airflow DAG for orchestrating an ETL workflow through sequential tasks for pipeline initialization, data extraction, and successful pipeline completion.		
7	To design and implement an ETL pipeline using Python for extracting data from CSV files, cleaning and transforming the data, validating the records, and loading the processed data into a database.		
8	To implement PySpark DataFrame operations for reading and analyzing CSV data, including schema inspection, filtering, grouping, aggregation, duplicate removal, joining datasets, and calculating average sales by product category.		
9	To design and implement an end-to-end data pipeline that extracts e-commerce data from CSV files, cleans and transforms the data using Python/Pandas, loads it into a SQL database, generates a report, and handles both historical and newly arriving data.		
Mini Project	To design and implement a retail data warehouse ETL pipeline in Python that ingests store, product and transaction data from CSV and JSON files, applies data-quality rules, shapes the data into a star schema, loads it incrementally into SQL Server, and generates a revenue report.		

Practical No. – 1

Aim:

To perform fundamental data handling and processing operations using Python, including:

1. Parsing and extracting data from text, CSV, HTML, XML and JSON files
2. Performing read and write operations on binary files
3. Searching, splitting and replacing strings using regular expressions
4. Creating and managing a relational database using SQL CRUD operations

1. Parsing and extracting data from text, CSV, HTML, XML, and JSON files

Code:

```
import csv
import json
import xml.etree.ElementTree as ET
from bs4 import BeautifulSoup
import pandas as pd
import io

def parse_and_check_csv(csv_data):
    print("--- Parsing CSV ---")
    try:
        df = pd.read_csv(io.StringIO(csv_data))
        print("Data loaded successfully:")
        display(df)

        missing_summary = df.isnull().sum()
        print("\nMissing values count per column:")
        print(missing_summary)

        duplicates = df.duplicated().sum()
        print(f"\nDuplicate rows found: {duplicates}")

        return df
    except Exception as e:
        print(f"Error parsing CSV: {e}")
        return None

def parse_and_check_json(json_data):
    print("\n--- Parsing JSON ---")
    try:
        data = json.loads(json_data)
        print("Valid JSON syntax parsed successfully.")

        if not data:
            print("Warning: The JSON document is empty.")
        else:
            print(f"Root element type: {type(data)}")
            if isinstance(data, list):
                for i, item in enumerate(data):
                    null_keys = [k for k, v in item.items() if v is None or v == ""]
                    if null_keys:
                        print(f"Warning: Record {i} has empty values in fields: {null_keys}")
        return data
    except json.JSONDecodeError as e:
        print(f"JSON Decode Error (Invalid Syntax): {e}")
        return None

def parse_and_check_html(html_data):
    print("\n--- Parsing HTML ---")
    try:
        soup = BeautifulSoup(html_data, 'html.parser')
        print("HTML parsed with BeautifulSoup.")

        tables = soup.find_all('table')
        print(f"Total tables found: {len(tables)}")
        for i, table in enumerate(tables):
            rows = table.find_all('tr')
            print(f"  Table {i+1} has {len(rows)} rows.")
```

```
            if len(rows) == 0:
                print(f"  Anomaly: Table {i+1} is empty.")
        return soup
    except Exception as e:
        print(f"Error parsing HTML: {e}")
        return None

def parse_and_check_xml(xml_data):
    print("\n--- Parsing XML ---")
    try:
        root = ET.fromstring(xml_data)
        print(f"XML root element tag: '{root.tag}' parsed successfully.")

        empty_elements = []
        for elem in root.iter():
            if elem.text is None or elem.text.strip() == "":
                if len(elem) == 0:
                    empty_elements.append(elem.tag)

        if empty_elements:
            print(f"Warning: Found empty leaf elements: {empty_elements}")
        else:
            print("No empty elements found in XML structure.")
        return root
    except ET.ParseError as e:
        print(f"XML Parse Error: {e}")
        return None

def parse_and_check_text(text_data):
    print("\n--- Parsing Plain Text ---")
    try:
        lines = text_data.strip().split('\n')
        print(f"Total lines parsed: {len(lines)}")

        empty_lines = [i+1 for i, line in enumerate(lines) if not line.strip()]
        if empty_lines:
            print(f"Warning: Found empty or blank lines at line number(s): {empty_lines}")
        return lines
    except Exception as e:
        print(f"Error parsing plain text: {e}")
        return None

csv_sample = """id,name,age,email
1,Alice,30,alice@example.com
2,Bob,,bob@example.com
3,Charlie,25,
1,Alice,30,alice@example.com"""

json_sample = """[
    {"id": 1, "name": "Product A", "price": 12.99},
    {"id": 2, "name": "Product B", "price": null},
    {"id": 3, "name": "", "price": 15.00}
]"""

html_sample = """<html>
<body>
    <h1>Product Summary</h1>
    <table>
        <tr><th>Item</th><th>Stock</th></tr>
        <tr><td>Item A</td><td>10</td></tr>
        <tr><td>Item B</td><td></td></tr>
    </table>
    <table id='empty_table'>
    </table>
</body>
</html>"""

xml_sample = """<store>
    <item>
        <id>101</id>
        <name>Laptop</name>
        <price>999</price>
    </item>
```

```
    <item>
        <id>102</id>
        <name></name>
        <price>499</price>
    </item>
</store>"""

text_sample = """First line of configuration data.
Second line contains value: 123.

Fourth line after an anomaly of an empty line.
"""

parse_and_check_csv(csv_sample)
parse_and_check_json(json_sample)
parse_and_check_html(html_sample)
parse_and_check_xml(xml_sample)
parse_and_check_text(text_sample)
```

Output:

```
--- Parsing CSV ---
Data loaded successfully:
   id     name   age              email
0   1    Alice  30.0  alice@example.com
1   2      Bob   NaN    bob@example.com
2   3  Charlie  25.0                NaN
3   1    Alice  30.0  alice@example.com

Missing values count per column:
id       0
name     0
age      1
email    1
dtype: int64

Duplicate rows found: 1

--- Parsing JSON ---
Valid JSON syntax parsed successfully.
Root element type: <class 'list'>
Warning: Record 1 has empty values in fields: ['price']
Warning: Record 2 has empty values in fields: ['name']

--- Parsing HTML ---
HTML parsed with BeautifulSoup.
Total tables found: 2
  Table 1 has 3 rows.
  Table 2 has 0 rows.
  Anomaly: Table 2 is empty.

--- Parsing XML ---
XML root element tag: 'store' parsed successfully.
Warning: Found empty leaf elements: ['name']

--- Parsing Plain Text ---
Total lines parsed: 4
Warning: Found empty or blank lines at line number(s): [3]
['First line of configuration data.',
 'Second line contains value: 123.',
 '',
 'Fourth line after an anomaly of an empty line.']
```

2. Performing read and write operations on binary files

Code:

```
import os

def write_binary_file(filepath, byte_data):
    """Writes binary data to a file safely, creating directory structures if needed."""
    print(f"Writing to binary file: {filepath}")
    try:
        dir_name = os.path.dirname(filepath)
        if dir_name and not os.path.exists(dir_name):
```

```
            os.makedirs(dir_name)

        with open(filepath, 'wb') as binary_file:
            binary_file.write(byte_data)
        print("Binary data written successfully.")
        return True
    except IOError as e:
        print(f"I/O Error during writing binary file: {e}")
        return False
    except Exception as e:
        print(f"Unexpected error during write: {e}")
        return False

def read_binary_file(filepath):
    """Reads binary data from a file safely, returning None if errors occur."""
    print(f"\nReading from binary file: {filepath}")
    if not os.path.exists(filepath):
        print(f"Error: The file '{filepath}' does not exist.")
        return None

    try:
        with open(filepath, 'rb') as binary_file:
            data = binary_file.read()
        print(f"Successfully read {len(data)} bytes.")
        return data
    except IOError as e:
        print(f"I/O Error during reading binary file: {e}")
        return None
    except Exception as e:
        print(f"Unexpected error during read: {e}")
        return None

test_filename = "demo_file.bin"
sample_bytes = bytes([120, 3, 255, 0, 18, 99, 104, 101, 99, 107])

if write_binary_file(test_filename, sample_bytes):
    retrieved_bytes = read_binary_file(test_filename)

    if retrieved_bytes == sample_bytes:
        print("SUCCESS: Retrieved binary content matches original byte data!")
    else:
        print("FAILURE: Content mismatch.")

if os.path.exists(test_filename):
    os.remove(test_filename)
```

Output:

```
Writing to binary file: demo_file.bin
Binary data written successfully.

Reading from binary file: demo_file.bin
Successfully read 10 bytes.
SUCCESS: Retrieved binary content matches original byte data!
```

3. Searching, splitting and replacing strings using regular expressions

Code:

```
import re

def find_patterns(pattern, text):
    """Searches for all occurrences of a pattern in the given text."""
    print(f"Searching for pattern '{pattern}'...")
    try:
        compiled_re = re.compile(pattern)
        matches = compiled_re.finditer(text)
```

```
        results = []
        for match in matches:
            results.append({
                "match": match.group(),
                "start_index": match.start(),
                "end_index": match.end()
            })

        if results:
            print(f"Found {len(results)} match(es):")
            for res in results:
                print(f" - '{res['match']}' at position
{res['start_index']}:{res['end_index']}")
        else:
            print("No matches found.")
        return results
    except re.error as e:
        print(f"Invalid Regular Expression: {e}")
        return []

def split_by_pattern(pattern, text):
    """Splits a string based on match occurrences of a pattern."""
    print(f"\nSplitting text using pattern '{pattern}'...")
    try:
        segments = re.split(pattern, text)
        # Remove empty string elements from results
        segments = [seg.strip() for seg in segments if seg.strip()]
        print(f"Splits generated: {segments}")
        return segments
    except re.error as e:
        print(f"Invalid Regular Expression: {e}")
        return []

def replace_pattern(pattern, replacement, text):
    """Replaces instances of a pattern with a replacement string/callback."""
    print(f"\nReplacing pattern '{pattern}' with '{replacement}'...")
    try:
        modified_text, replacement_count = re.subn(pattern, replacement, text)
        print(f"Replacements made: {replacement_count}")
        print(f"Result: '{modified_text}'")
        return modified_text
    except re.error as e:
        print(f"Invalid Regular Expression: {e}")
        return text

target_text = "Contact us at info@example.com or support@business-hub.co.uk before tomorrow!"

find_patterns(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', target_text)

split_by_pattern(r'[\s,!.@]+', target_text)

replace_pattern(r'\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b', '[REDACTED_EMAIL]',
target_text)
```

Output:

```
Searching for pattern '\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b'...
Found 2 match(es):
 - 'info@example.com' at position 14:30
 - 'support@business-hub.co.uk' at position 34:60

Splitting text using pattern '[\s,!.@]+'...
Splits generated: ['Contact', 'us', 'at', 'info', 'example', 'com', 'or', 'support', 'business-hub', 'co'
    , 'uk', 'before', 'tomorrow']

Replacing pattern '\b[A-Za-z0-9._%+-]+@[A-Za-z0-9.-]+\.[A-Z|a-z]{2,}\b' with '[REDACTED_EMAIL]'...
Replacements made: 2
Result: 'Contact us at [REDACTED_EMAIL] or [REDACTED_EMAIL] before tomorrow!'
'Contact us at [REDACTED_EMAIL] or [REDACTED_EMAIL] before tomorrow!'
```

4. Creating and managing a relational database using SQL CRUD operations

SQL Script (SQL Server):

```
CREATE DATABASE BookstoreApp;
GO
USE BookstoreApp;
GO

CREATE TABLE Books (
    book_id INT PRIMARY KEY,
    title VARCHAR(100) NOT NULL,
    author VARCHAR(50) NOT NULL,
    price DECIMAL(5, 2),
    stock_quantity INT
);

INSERT INTO Books (book_id, title, author, price, stock_quantity)
VALUES
    (1, 'Deep Learning', 'Ian Goodfellow', 45.99, 10),
    (2, 'Python Crash Course', 'Eric Matthes', 25.50, 20),
    (3, 'SQL QuickStart', 'Walter Shields', 19.99, 15),
    (4, 'The Pragmatic Programmer', 'Andy Hunt', 35.00, 5);

SELECT * FROM Books;
UPDATE Books
SET stock_quantity = 14
WHERE book_id = 3;
DELETE FROM Books
WHERE book_id = 4;
```

Output (result of SELECT * FROM Books):

	book_id	title	author	price	stock_quantity
1	1	Deep Learning	Ian Goodfellow	45.99	10
2	2	Python Crash Course	Eric Matthes	25.50	20
3	3	SQL QuickStart	Walter Shields	19.99	15
4	4	The Pragmatic Programmer	Andy Hunt	35.00	5

Practical No. – 2

Aim:

To perform the ETL process in Power BI by extracting, transforming, and loading data to prepare it for analysis.

Step 1 — SQL Server: create the SalesDW star schema

SQL Script:

```
CREATE DATABASE SalesDW;
GO
USE SalesDW;
GO

CREATE TABLE DimProduct (
    ProductKey INT IDENTITY(1,1) PRIMARY KEY,
    ProductID INT NOT NULL,
    ProductName NVARCHAR(100) NOT NULL,
    QuantityPerUnit NVARCHAR(50)
);

CREATE TABLE DimDate (
    DateKey INT PRIMARY KEY,
    FullDate DATE,
    OrderYear INT,
    OrderMonth INT
);

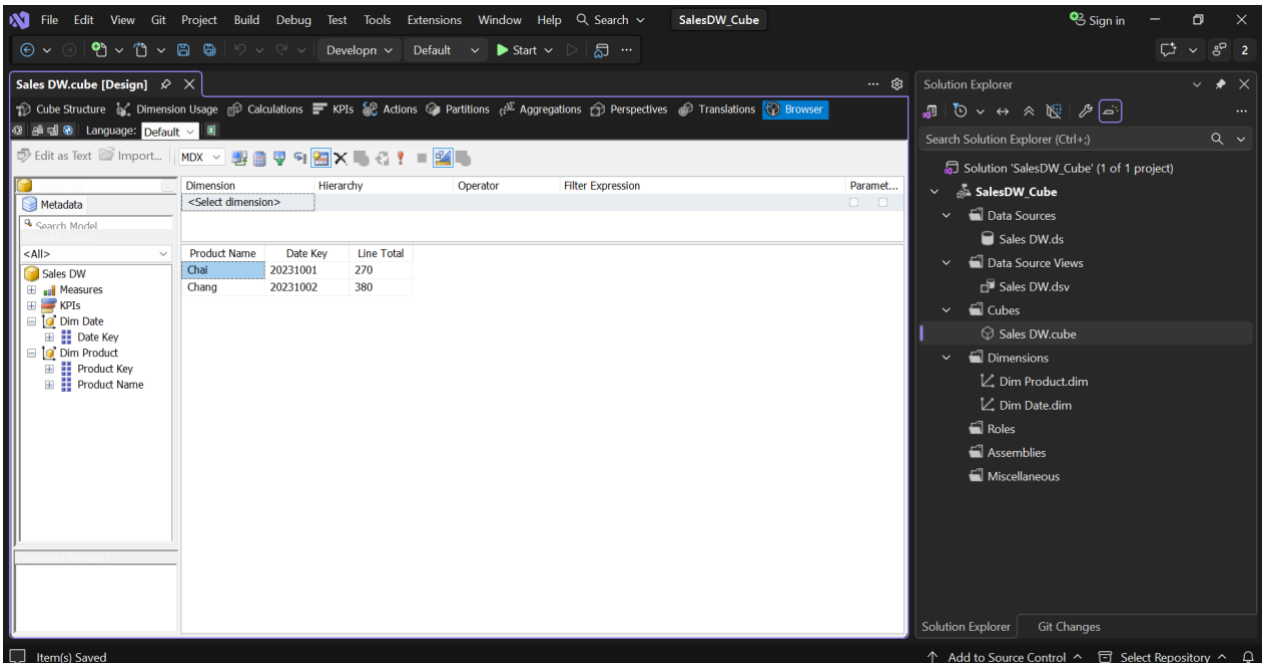
CREATE TABLE FactSales (
    SalesKey INT IDENTITY(1,1) PRIMARY KEY,
    ProductKey INT FOREIGN KEY REFERENCES DimProduct(ProductKey),
    DateKey INT FOREIGN KEY REFERENCES DimDate(DateKey),
    Quantity INT,
    UnitPrice DECIMAL(18,2),
    LineTotal DECIMAL(18,2)
);

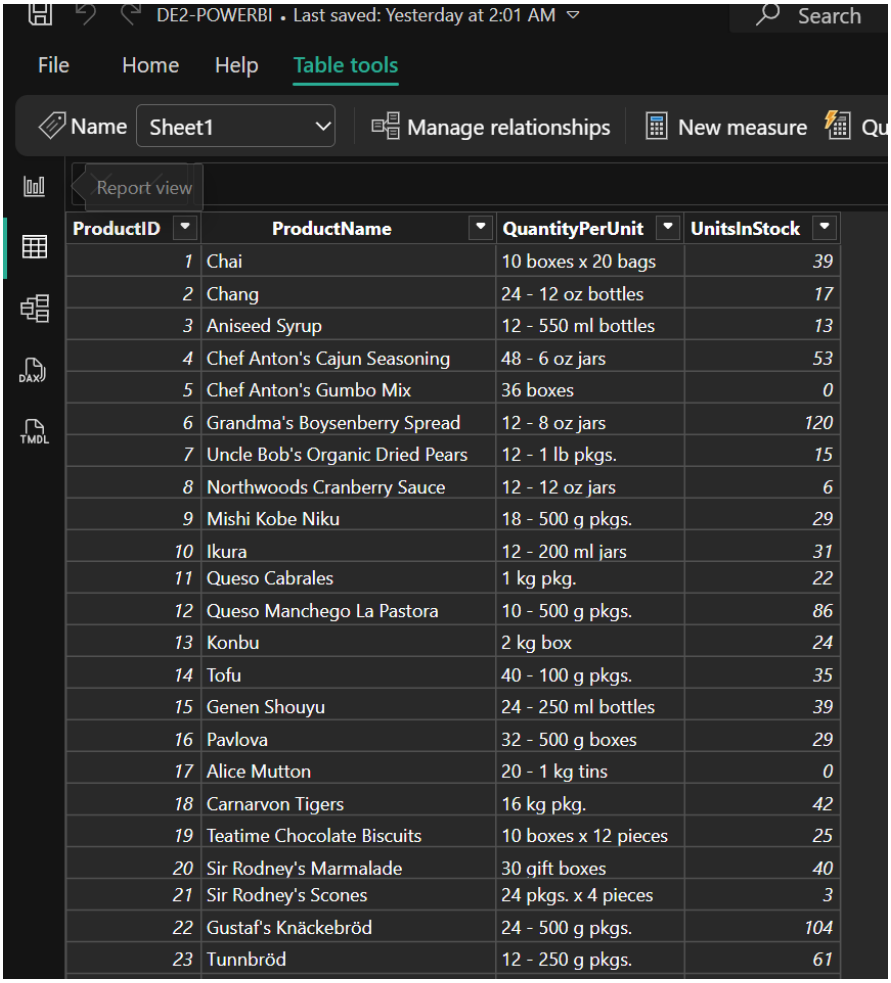
INSERT INTO DimProduct (ProductID, ProductName, QuantityPerUnit)
VALUES
(1, 'Chai', '10 boxes x 20 bags'),
(2, 'Chang', '24 - 12 oz bottles');

INSERT INTO DimDate (DateKey, FullDate, OrderYear, OrderMonth)
VALUES
(20231001, '2023-10-01', 2023, 10),
(20231002, '2023-10-02', 2023, 10);

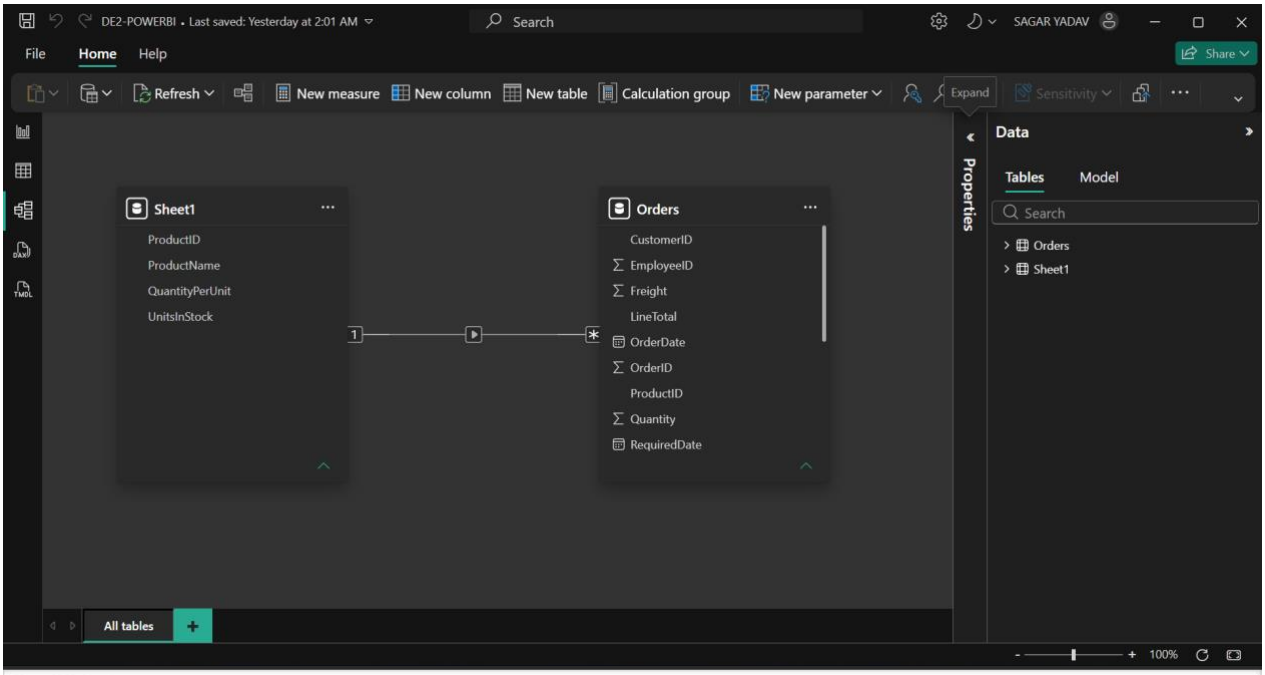
INSERT INTO FactSales (ProductKey, DateKey, Quantity, UnitPrice, LineTotal)
VALUES
(1, 20231001, 15, 18.00, 270.00),
(2, 20231002, 20, 19.00, 380.00);
```

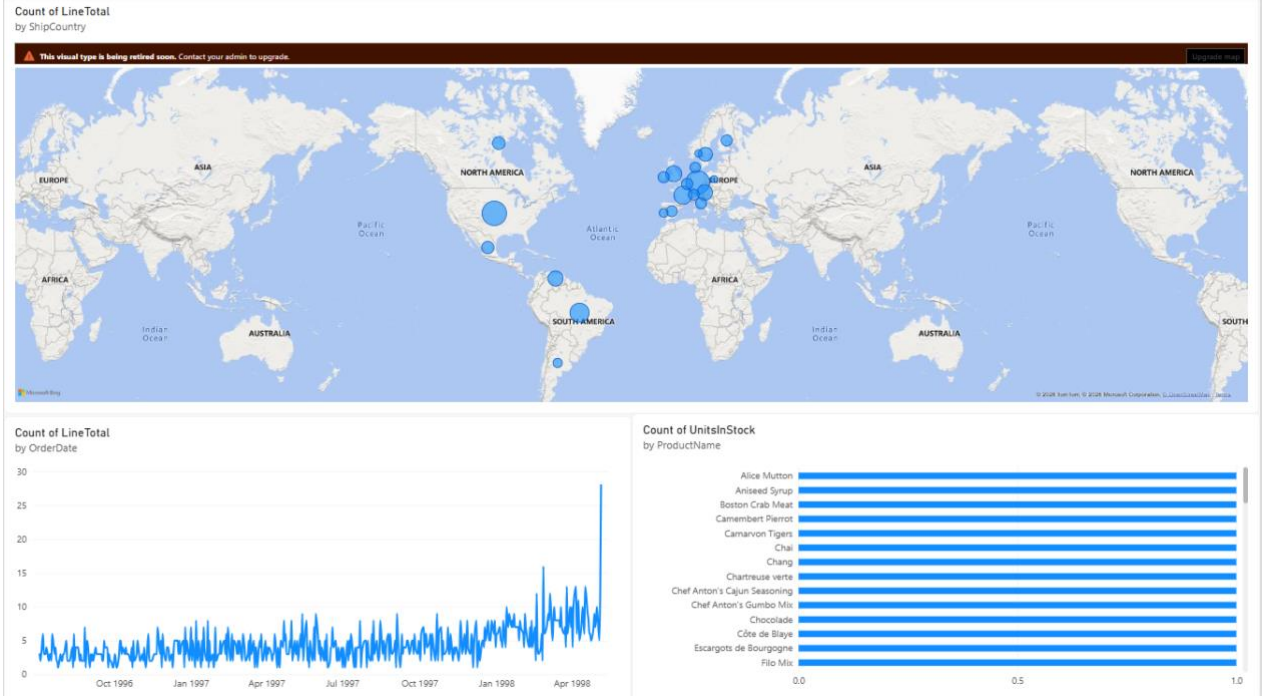
Output:





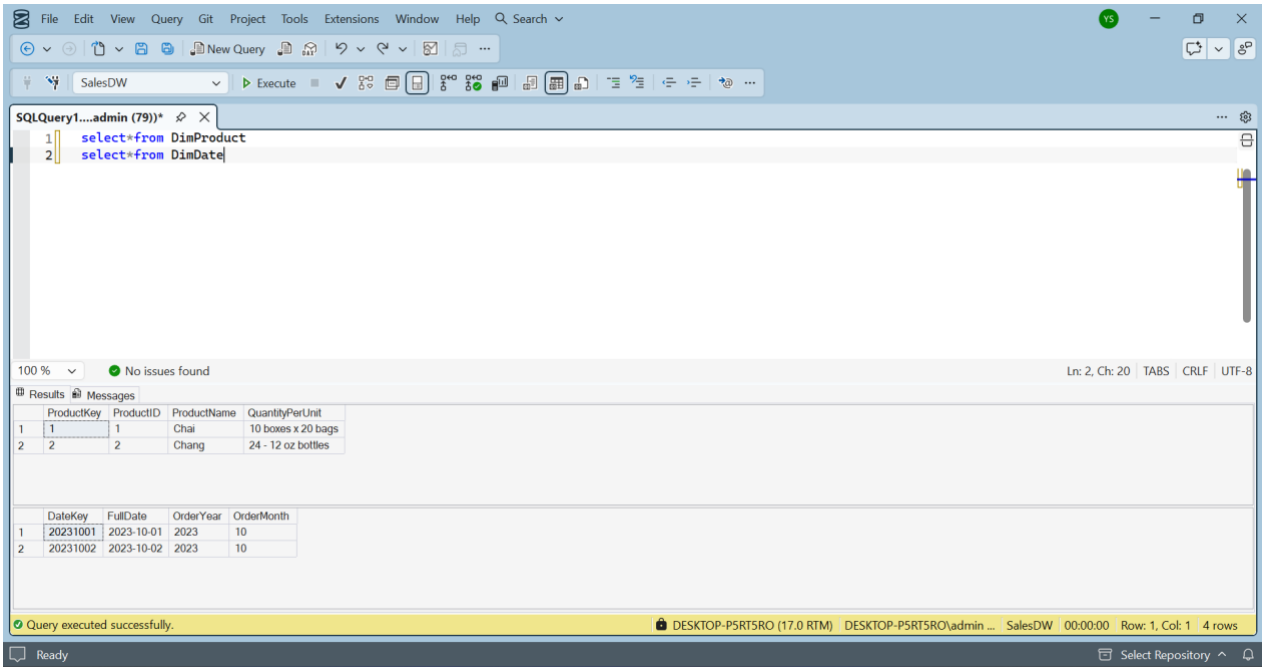
ProductID	ProductName	QuantityPerUnit	UnitsInStock
1	Chai	10 boxes x 20 bags	39
2	Chang	24 - 12 oz bottles	17
3	Aniseed Syrup	12 - 550 ml bottles	13
4	Chef Anton's Cajun Seasoning	48 - 6 oz jars	53
5	Chef Anton's Gumbo Mix	36 boxes	0
6	Grandma's Boysenberry Spread	12 - 8 oz jars	120
7	Uncle Bob's Organic Dried Pears	12 - 1 lb pkgs.	15
8	Northwoods Cranberry Sauce	12 - 12 oz jars	6
9	Mishi Kobe Niku	18 - 500 g pkgs.	29
10	Ikura	12 - 200 ml jars	31
11	Queso Cabrales	1 kg pkg.	22
12	Queso Manchego La Pastora	10 - 500 g pkgs.	86
13	Konbu	2 kg box	24
14	Tofu	40 - 100 g pkgs.	35
15	Genen Shouyu	24 - 250 ml bottles	39
16	Pavlova	32 - 500 g boxes	29
17	Alice Mutton	20 - 1 kg tins	0
18	Carnarvon Tigers	16 kg pkg.	42
19	Teatime Chocolate Biscuits	10 boxes x 12 pieces	25
20	Sir Rodney's Marmalade	30 gift boxes	40
21	Sir Rodney's Scones	24 pkgs. x 4 pieces	3
22	Gustaf's Knäckebröd	24 - 500 g pkgs.	104
23	Tunnbröd	12 - 250 g pkgs.	61

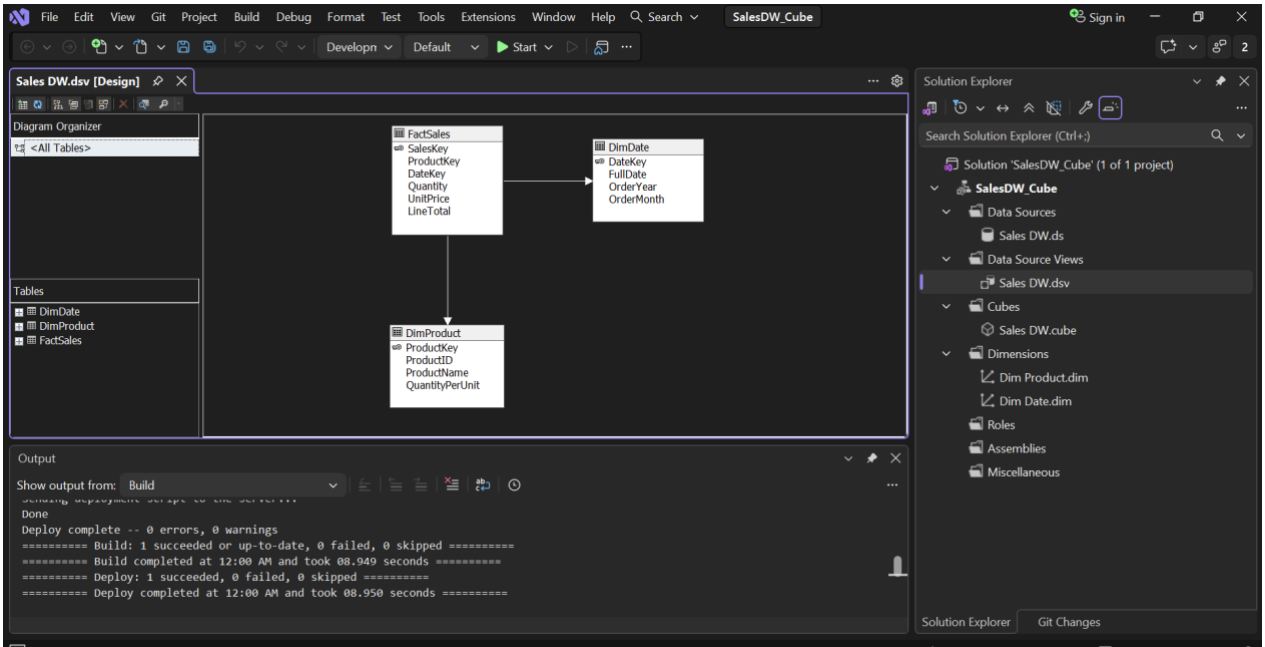

DE2-POWERBI • Last saved: Yesterday at 2:01 AM
Search
SAGAR YADAV
File
Home
Help
Share
Refresh
New measure
New column
New table
Calculation group
New parameter
Expand
Sensitivity
Data
Tables
Model
Search
Orders
Sheet1
Properties
Sheet1
ProductID
ProductName
QuantityPerUnit
UnitsInStock
Orders
CustomerID
EmployeeID
Freight
LineTotal
OrderDate
OrderID
ProductID
Quantity
RequiredDate
All tables
100%

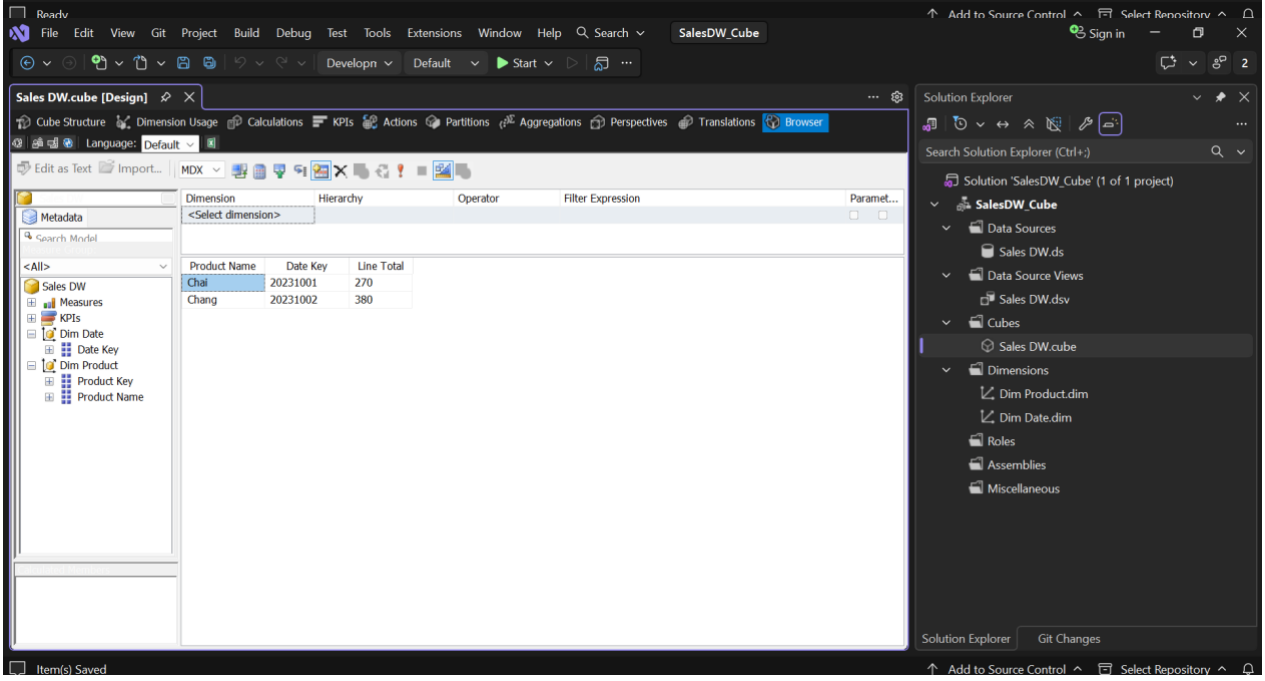

Count of LineTotal
by ShipCountry
This visual type is being retired soon. Contact your admin to upgrade.
NORTH AMERICA
EUROPE
ASIA
AFRICA
SOUTH AMERICA
AUSTRALIA
Pacific Ocean
Atlantic Ocean
Indian Ocean

Count of LineTotal
by OrderDate
30
25
20
15
10
5
0
Oct 1996
Jan 1997
Apr 1997
Jul 1997
Oct 1997
Jan 1998
Apr 1998

Count of UnitsInStock
by ProductName
Alice Mutton
Aniseed Syrup
Boston Crab Meat
Camembert Pierrot
Carnarvon Tigers
Chai
Chang
Chartreuse verte
Chef Anton's Cajun Seasoning
Chef Anton's Gumbo Mix
Chocolade
Côte de Blaye
Escargots de Bourgogne
Filo Mix
0.0
0.5
1.0


SalesDW
Execute
SQLQuery1....admin (79))*
select*from DimProduct
select*from DimDate
No issues found
Results
Messages
ProductKey ProductID ProductName QuantityPerUnit
1 1 1 Chai 10 boxes x 20 bags
2 2 2 Chang 24 - 12 oz bottles
DateKey FullDate OrderYear OrderMonth
1 20231001 2023-10-01 2023 10
2 20231002 2023-10-02 2023 10
Query executed successfully.
DESKTOP-P5RT5RO (17.0 RTM)
DESKTOP-P5RT5RO\admin ...
SalesDW
00:00:00
Row: 1, Col: 1
4 rows
Ready


SalesDW_Cube
Sales DW.dsv [Design]
Diagram Organizer
<All Tables>
FactSales
SalesKey
ProductKey
DateKey
Quantity
UnitPrice
LineTotal
DimDate
DateKey
FullDate
OrderYear
OrderMonth
DimProduct
ProductKey
ProductID
ProductName
QuantityPerUnit
Tables
DimDate
DimProduct
FactSales
Output
Show output from: Build
Done
Deploy complete -- 0 errors, 0 warnings
========== Build: 1 succeeded or up-to-date, 0 failed, 0 skipped ==========
========== Build completed at 12:00 AM and took 08.949 seconds ==========
========== Deploy: 1 succeeded, 0 failed, 0 skipped ==========
========== Deploy completed at 12:00 AM and took 08.950 seconds ==========
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'SalesDW_Cube' (1 of 1 project)
SalesDW_Cube
Data Sources
Sales DW.ds
Data Source Views
Sales DW.dsv
Cubes
Sales DW.cube
Dimensions
Dim Product.dim
Dim Date.dim
Roles
Assemblies
Miscellaneous
Solution Explorer
Git Changes
Ready
Add to Source Control
Select Repository


SalesDW_Cube
Sales DW.cube [Design]
Cube Structure
Dimension Usage
Calculations
KPIs
Actions
Partitions
Aggregations
Perspectives
Translations
Browser
Language: Default
Edit as Text
Import...
MDX
Dimension
Hierarchy
Operator
Filter Expression
Paramet...
<Select dimension>
Metadata
Search Model
<All>
Sales DW
Measures
KPIs
Dim Date
Date Key
Dim Product
Product Key
Product Name
Product Name Date Key Line Total
Chai 20231001 270
Chang 20231002 380
Solution Explorer
Search Solution Explorer (Ctrl+;)
Solution 'SalesDW_Cube' (1 of 1 project)
SalesDW_Cube
Data Sources
Sales DW.ds
Data Source Views
Sales DW.dsv
Cubes
Sales DW.cube
Dimensions
Dim Product.dim
Dim Date.dim
Roles
Assemblies
Miscellaneous
Solution Explorer
Git Changes
Item(s) Saved
Add to Source Control
Select Repository

Practical No. – 3

Aim:

To implement and demonstrate single and multiple document insertion and retrieval operations in MongoDB.

Step 1: Insert Single Document

Query:

```
db.items.insertOne({ name: "laptop", price: 999 });
```

Step 2: View Inserted Document

Query:

```
db.items.find();
```

Step 3: Insert Multiple Documents

Query:

```
db.products.insertMany([
  { name: "phone", price: 500, stock: 10 },
  { name: "tablet", price: 300, stock: 5 },
  { name: "watch", price: 150, stock: 0 }
]);
```

Step 4: View Products in Formatted JSON

Query:

```
db.products.find().pretty();
```

Output:

```
C:\Users\admin>mongosh
Current Mongosh Log ID: 6ac20ebb1f008ba9740339db
Connecting to:          mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:          9.0.2
Using Mongosh:          2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/


To help improve our products, pseudo-anonymous usage data is collected from interactive and agentic sessions only, and sent to MongoD
B periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

------
   The server generated these startup warnings when booting
   2026-10-04T00:18:07.551+05:30: Access control is not enabled for the database. Read and write access to data and configuration is
unrestricted
------

test> db.items.insertOne({ name: "laptop", price: 999 });
{
  acknowledged: true,
  insertedId: ObjectId('6ac20ee41f008ba9740339dc')
}
test> db.items.find();
[
  {
    _id: ObjectId('6ac20ee41f008ba9740339dc'),
    name: 'laptop',
    price: 999
  }
]
test> db.products.insertMany([ { name: "phone", price: 500, stocstock: 10 }, { name: "tablet", price: 300, stock: 5 }, { name: "watch
  }
]
test> db.products.insertMany([ { name: "phone", price: 500, stocstock: 10 }, { name: "tablet", price: 300, stock: 5 }, { name: "watch
", price: 150, stock: 0 } ]);
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac20f021f008ba9740339dd'),
    '1': ObjectId('6ac20f021f008ba9740339de'),
    '2': ObjectId('6ac20f021f008ba9740339df')
  }
}
test> db.products.find().pretty();
[
  {
    _id: ObjectId('6ac20f021f008ba9740339dd'),
    name: 'phone',
    price: 500,
    stock: 10
  },
  {
    _id: ObjectId('6ac20f021f008ba9740339de'),
    name: 'tablet',
    price: 300,
    stock: 5
  },
  {
    _id: ObjectId('6ac20f021f008ba9740339df'),
    name: 'watch',
    price: 150,
    stock: 0
  }
]
```

Practical No. – 4

Aim:

To perform noise elimination, feature selection, and exploratory data analysis (EDA) using Python libraries.

Part 1 — Data cleaning: missing values, duplicates and normalization

Code:

```
import pandas as pd

data = {
    'Name': ['Amit', 'Riya', 'Amit', 'Neha', None],
    'Age': [22, 24, 22, None, 26],
    'Marks': [85, 90, 85, 78, None]
}

df = pd.DataFrame(data)

print("Original Data:")
print(df)

print("\nHandling Missing Values...")

df['Age'] = df['Age'].fillna(df['Age'].mean())

df['Marks'] = df['Marks'].fillna(df['Marks'].mean())

df['Name'] = df['Name'].fillna('Unknown')

print(df)

print("\nRemoving Duplicates...")

df = df.drop_duplicates()

print(df)

print("\nNormalizing Age and Marks...")

df['Age_Normalized'] = (df['Age'] - df['Age'].min()) / (df['Age'].max() - df['Age'].min())

df['Marks_Normalized'] = (df['Marks'] - df['Marks'].min()) / (df['Marks'].max() -
df['Marks'].min())

print(df)

print("\nFinal Processed Data:")
print(df)
```

Output:

```
Original Data:
   Name   Age  Marks
0  Amit  22.0   85.0
1  Riya  24.0   90.0
2  Amit  22.0   85.0
3  Neha   NaN   78.0
4  None  26.0    NaN

Handling Missing Values...
      Name   Age  Marks
0     Amit  22.0   85.0
1     Riya  24.0   90.0
2     Amit  22.0   85.0
3     Neha  23.5   78.0
4  Unknown  26.0   84.5

Removing Duplicates...
      Name   Age  Marks
0     Amit  22.0   85.0
1     Riya  24.0   90.0
3     Neha  23.5   78.0
4  Unknown  26.0   84.5

Normalizing Age and Marks...
      Name   Age  Marks  Age_Normalized  Marks_Normalized
0     Amit  22.0   85.0           0.000          0.583333
1     Riya  24.0   90.0           0.500          1.000000
3     Neha  23.5   78.0           0.375          0.000000
4  Unknown  26.0   84.5           1.000          0.541667

Final Processed Data:
      Name   Age  Marks  Age_Normalized  Marks_Normalized
0     Amit  22.0   85.0           0.000          0.583333
1     Riya  24.0   90.0           0.500          1.000000
3     Neha  23.5   78.0           0.375          0.000000
4  Unknown  26.0   84.5           1.000          0.541667
```

Part 2 — Noise elimination, feature selection and EDA

Code:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.feature_selection import VarianceThreshold

data = {
    'Age': [20, 21, 22, 23, 24, 100],
    'Marks': [75, 80, 85, 90, 95, 20],
    'Attendance': [85, 88, 90, 92, 94, 50],
    'Constant': [1, 1, 1, 1, 1, 1]
}

df = pd.DataFrame(data)
```

```
print("Original Dataset:")
print(df)

Q1 = df['Age'].quantile(0.25)
Q3 = df['Age'].quantile(0.75)
IQR = Q3 - Q1

lower = Q1 - 1.5 * IQR
upper = Q3 + 1.5 * IQR

df_clean = df[(df['Age'] >= lower) & (df['Age'] <= upper)]

print("\nDataset after Noise Elimination:")
print(df_clean)

selector = VarianceThreshold(threshold=0.0)
selected = selector.fit_transform(df_clean)

selected_columns = df_clean.columns[selector.get_support()]

print("\nSelected Features:")
print(selected_columns)

print("\nDataset Information:")
print(df_clean.info())

print("\nStatistical Summary:")
print(df_clean.describe())

print("\nCorrelation Matrix:")
print(df_clean.corr())

df_clean.hist(figsize=(8,6))
plt.suptitle("Histogram of Features")
plt.show()

df_clean.boxplot(figsize=(8,5))
plt.title("Boxplot of Dataset")
plt.show()
```

Output:

```
Original Dataset:
   Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1
5  100     20          50         1

Dataset after Noise Elimination:
   Age  Marks  Attendance  Constant
0   20     75          85         1
1   21     80          88         1
2   22     85          90         1
3   23     90          92         1
4   24     95          94         1

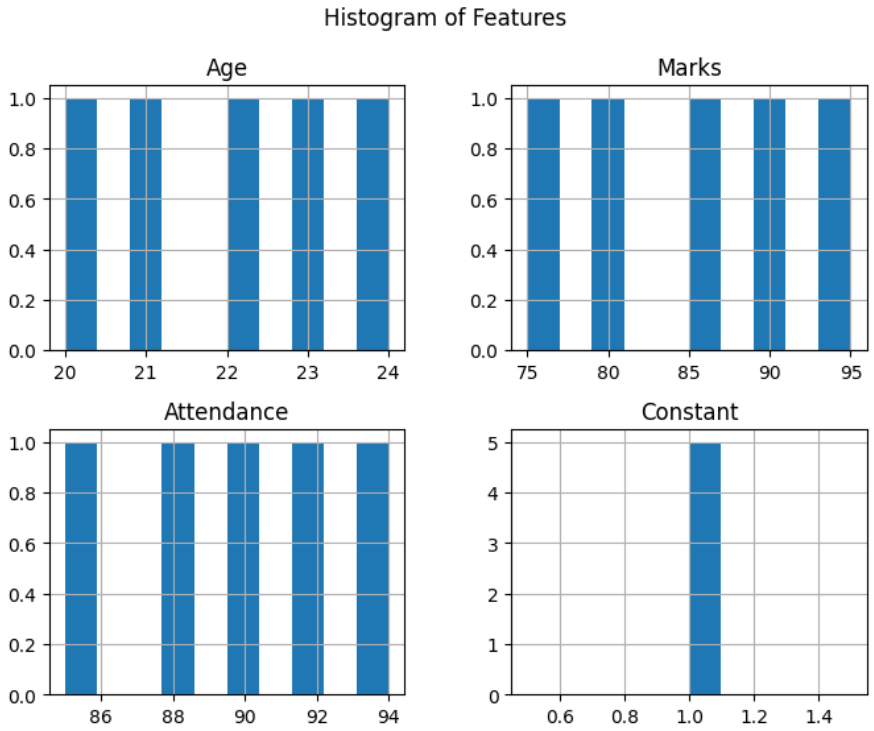
Selected Features:
Index(['Age', 'Marks', 'Attendance'], dtype='object')

Dataset Information:
<class 'pandas.core.frame.DataFrame'>
Index: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   Age         5 non-null      int64
 1   Marks       5 non-null      int64
 2   Attendance  5 non-null      int64
 3   Constant    5 non-null      int64
dtypes: int64(4)
memory usage: 200.0 bytes
None

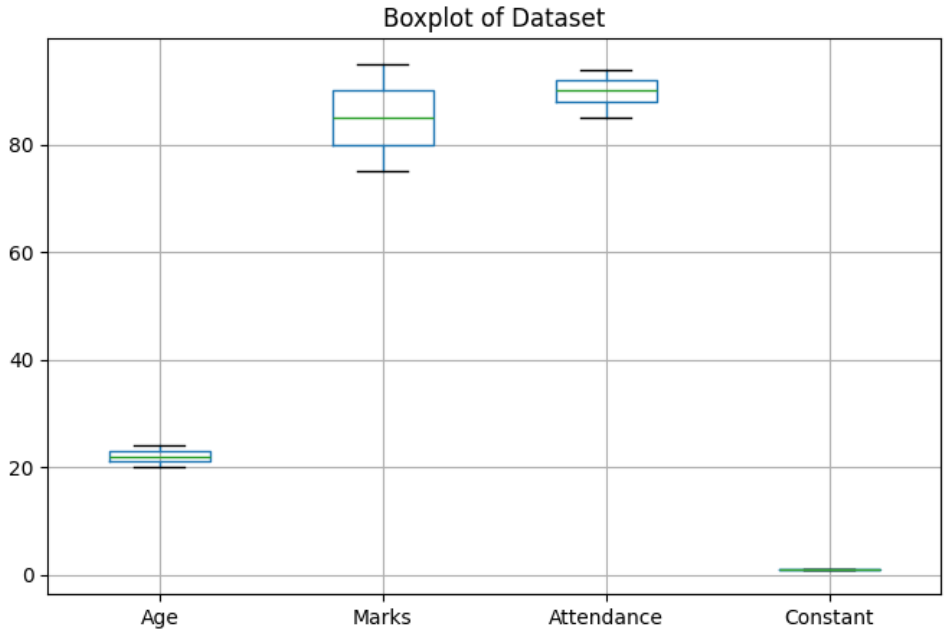
Statistical Summary:
             Age      Marks  Attendance  Constant
count   5.000000   5.000000     5.00000       5.0
mean   22.000000  85.000000    89.80000       1.0
std     1.581139   7.905694     3.49285       0.0
min    20.000000  75.000000    85.00000       1.0
25%    21.000000  80.000000    88.00000       1.0
50%    22.000000  85.000000    90.00000       1.0
75%    23.000000  90.000000    92.00000       1.0
max    24.000000  95.000000    94.00000       1.0

Correlation Matrix:
                Age     Marks  Attendance  Constant
Age         1.000000  1.000000    0.995893       NaN
Marks       1.000000  1.000000    0.995893       NaN
Attendance  0.995893  0.995893    1.000000       NaN
Constant         NaN       NaN         NaN       NaN
```

Histogram:



Boxplot:



Practical No. – 5

Aim:

To extract user data from a REST API and a flat CSV file, transform and merge the data using Python.

Code:

```
import pandas as pd
import requests

def extract_api_data(url):
    try:
        response = requests.get(url, timeout=10)
        response.raise_for_status()
        data = response.json()
        return pd.json_normalize(data)
    except requests.exceptions.RequestException as e:
        print(f"API Error: {e}")
        return pd.DataFrame()

def extract_csv_data(file_path):
    try:
        return pd.read_csv(file_path)
    except FileNotFoundError as e:
        print(f"File Error: {e}")
        return pd.DataFrame()
```

```
api_url = "https://jsonplaceholder.typicode.com/users"
api_df = extract_api_data(api_url)
if not api_df.empty:
    api_df = api_df[["id", "name", "email", "company.name"]]
    api_df.rename(columns={"company.name": "company"}, inplace=True)

csv_file = "locations.csv"
pd.DataFrame(
    {
        "id": list(range(1, 11)),
        "city": [
            "New York",
            "London",
            "Paris",
            "Tokyo",
            "Berlin",
            "Delhi",
            "Sydney",
            "Moscow",
            "Cairo",
            "Beijing",
        ],
    }
).to_csv(csv_file, index=False)

csv_df = extract_csv_data(csv_file)

if not api_df.empty and not csv_df.empty:
    merged_df = pd.merge(api_df, csv_df, on="id", how="inner")
    print("\n--- Merged ETL Pipeline Data View ---")
    display(merged_df.head())
    merged_df.to_csv("cleaned_warehouse_profiles.csv", index=False)
    print("\nData successfully saved to 'cleaned_warehouse_profiles.csv'")
```

Output:

```
--- Merged ETL Pipeline Data View ---
   id              name                         email             company  \
0   1     Leanne Graham          Sincere@april.biz     Romaguera-Crona
1   2      Ervin Howell          Shanna@melissa.tv        Deckow-Crist
2   3  Clementine Bauch         Nathan@yesenia.net  Romaguera-Jacobson
3   4  Patricia Lebsack  Julianne.OConner@kory.org       Robel-Corkery
4   5  Chelsey Dietrich   Lucio_Hettinger@annie.ca         Keebler LLC

       city
0  New York
1    London
2     Paris
3     Tokyo
4    Berlin

Data successfully saved to 'cleaned_warehouse_profiles.csv'
```

Practical No. – 6

Aim:

To design and configure a scheduled Apache Airflow DAG for orchestrating an ETL workflow through sequential tasks for pipeline initialization, data extraction, and successful pipeline completion.

Code:

```
from datetime import datetime, timedelta
from airflow import DAG
from airflow.providers.standard.operators.bash import BashOperator
from airflow.providers.standard.operators.empty import EmptyOperator

default_args = {
    "owner": "data_engineering_lab",
    "depends_on_past": False,
    "start_date": datetime(2026, 1, 1),
    "email_on_failure": False,
    "email_on_retry": False,
    "retries": 1,
    "retry_delay": timedelta(minutes=5),
}

with DAG(
    "university_etl_orchestration",
    default_args=default_args,
    description="Lab assignment for API and Flat File ETL orchestration",
    schedule=timedelta(days=1),
    catchup=False,
) as dag:

    start_pipeline = EmptyOperator(task_id="start_pipeline")

    execute_extraction = BashOperator(
        task_id="run_extraction_script",
        bash_command="python3 /absolute/path/to/data_extraction.py",
    )

    pipeline_complete = BashOperator(
        task_id="log_pipeline_success",
        bash_command='echo "ETL Execution completed successfully at $(date)"',
    )

    start_pipeline >> execute_extraction >> pipeline_complete
```

Task execution test:

```
# We test the tasks sequentially inside the notebook context
# to bypass metadata database serialization constraints in Airflow 3
print("Executing 'start_pipeline'...")
start_pipeline.execute(context={})

print("\nExecuting 'run_extraction_script'...")
try:
    execute_extraction.execute(context={})
except Exception as e:
    print(f"Task execution failed as expected (since file doesn't exist): {e}")

print("\nExecuting 'log_pipeline_success'...")
pipeline_complete.execute(context={})
```

Output:

```
Executing 'start_pipeline'...
2026-10-03T20:09:30.195998Z [warning  ] EmptyOperator.execute cannot be called outside of the Task Runner
    ! [airflow.task.operators.airflow.providers.standard.operators.empty.EmptyOperator] loc=operator.py:4
    56

Executing 'run_extraction_script'...
2026-10-03T20:09:30.199116Z [warning  ] BashOperator.execute cannot be called outside of the Task Runner!
     [airflow.task.operators.airflow.providers.standard.operators.bash.BashOperator] loc=operator.py:456
2026-10-03T20:09:30.200475Z [info     ] Tmp dir root location: /tmp    [airflow.task.hooks.airflow.provid
    ers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:78
2026-10-03T20:09:30.203185Z [info     ] Running command: ['/usr/bin/bash', '-c', 'python3 /absolute/path/
    to/data_extraction.py'] [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHoo
    k] loc=subprocess.py:80
2026-10-03T20:09:30.205703Z [info     ] Output:                        [airflow.task.hooks.airflow.provid
    ers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:92
2026-10-03T20:09:30.236325Z [info     ] python3: can't open file '/absolute/path/to/data_extraction.py':
    [Errno 2] No such file or directory [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.S
    ubprocessHook] loc=subprocess.py:99
2026-10-03T20:09:30.244057Z [info     ] Command exited with return code 2 [airflow.task.hooks.airflow.pro
    viders.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:103
Task execution failed as expected (since file doesn't exist): Bash command failed. The command returned a
     non-zero exit code 2.

Executing 'log_pipeline_success'...
2026-10-03T20:09:30.245987Z [warning  ] BashOperator.execute cannot be called outside of the Task Runner!
     [airflow.task.operators.airflow.providers.standard.operators.bash.BashOperator] loc=operator.py:456
2026-10-03T20:09:30.247895Z [info     ] Tmp dir root location: /tmp    [airflow.task.hooks.airflow.provid
    ers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:78
2026-10-03T20:09:30.248810Z [info     ] Running command: ['/usr/bin/bash', '-c', 'echo "ETL Execution com
    pleted successfully at $(date)"'] [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.Sub
    processHook] loc=subprocess.py:80
2026-10-03T20:09:30.251329Z [info     ] Output:                        [airflow.task.hooks.airflow.provid
    ers.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:92
2026-10-03T20:09:30.259128Z [info     ] ETL Execution completed successfully at Sat Oct  3 08:09:30 PM UT
    C 2026 [airflow.task.hooks.airflow.providers.standard.hooks.subprocess.SubprocessHook] loc=subprocess
    .py:99
2026-10-03T20:09:30.261165Z [info     ] Command exited with return code 0 [airflow.task.hooks.airflow.pro
    viders.standard.hooks.subprocess.SubprocessHook] loc=subprocess.py:103
'ETL Execution completed successfully at Sat Oct  3 08:09:30 PM UTC 2026'
```

Practical No. – 7

Aim:

To design and implement an ETL pipeline using Python for extracting data from CSV files, cleaning and transforming the data, validating the records, and loading the processed data into a database.

Input files (raw_data):

sales_jan.csv

```
transaction_id,category,price,quantity,transaction_date
101,electronics,250.50,2,2023-01-15
102,clothing,-15.00,1,2023-01-16
103,furniture,120.00,4,2023-01-17
```

sales_feb.csv

```
transaction_id,category,price,quantity,transaction_date
104,electronics,899.99,1,2023-02-10
,groceries,5.50,10,2023-02-11
105,books,45.00,3,2023-02-12
```

sales_mar.json

```
[
  {"transaction_id": 106, "category": "sports", "price": 55.00, "quantity": 2,
"transaction_date": "2023-03-05"},
  {"transaction_id": 107, "category": "clothing", "price": 40.00, "quantity": 1,
"transaction_date": "2023-03-08"}
]
```

Code:

```
import pandas as pd
import glob
from sqlalchemy import create_engine
from sqlalchemy.engine import URL

def run_etl():
    print("--- Starting ETL Pipeline ---")

    print("[1/4] Extracting data from CSV and JSON sources...")

    csv_files = glob.glob("raw_data/*.csv")
    df_csv = pd.concat((pd.read_csv(f) for f in csv_files), ignore_index=True)

    df_json = pd.read_json("raw_data/sales_mar.json")

    df_staging = pd.concat([df_csv, df_json], ignore_index=True)
    print(f"Extracted {len(df_staging)} total records.")

    print("[2/4] Validating and removing invalid records...")

    df_valid = df_staging.dropna(subset=['transaction_id'])

    df_valid = df_valid[df_valid['price'] > 0]

    dropped_count = len(df_staging) - len(df_valid)
    print(f"Removed {dropped_count} invalid records.")

    print("[3/4] Transforming fields...")

    df_valid['category'] = df_valid['category'].str.upper()

    df_valid['transaction_date'] = pd.to_datetime(df_valid['transaction_date'])
```

```
    df_valid['total_value'] = df_valid['price'] * df_valid['quantity']

    df_valid['transaction_id'] = df_valid['transaction_id'].astype(int)

    print("[4/4] Executing incremental load to MS SQL Server...")

    server_name = r'DESKTOP-P5RT5RO'
    database_name = 'DataEngineeringLab'
    table_name = 'Fact_Sales'

    connection_url = URL.create(
        "mssql+pyodbc",
        host=server_name,
        database=database_name,
        query={
            "driver": "ODBC Driver 17 for SQL Server",
            "Trusted_Connection": "yes"
        }
    )

    engine = create_engine(connection_url, fast_executemany=True)

    try:
        query = f'SELECT transaction_id FROM {table_name}'
        existing_ids = pd.read_sql(query, engine)['transaction_id'].tolist()

        df_new = df_valid[~df_valid['transaction_id'].isin(existing_ids)]
    except Exception:
        df_new = df_valid

    if not df_new.empty:
        df_new.to_sql(table_name, engine, if_exists='append', index=False)
        print(f"SUCCESS: {len(df_new)} new records loaded into '{table_name}'.")
    else:
        print("SUCCESS: 0 new records to load. Database is up to date.")

    print("--- ETL Pipeline Complete ---")

if __name__ == "__main__":
    run_etl()
```

Output:

```
--- Starting ETL Pipeline ---
[1/4] Extracting data from CSV and JSON sources...
Extracted 8 total records.
[2/4] Validating and removing invalid records...
Removed 2 invalid records.
[3/4] Transforming fields...
[4/4] Executing incremental load to MS SQL Server...
SUCCESS: 6 new records loaded into 'Fact_Sales'.
--- ETL Pipeline Complete ---
```

SQL Script (SQL Server):

```
CREATE DATABASE DataEngineeringLab;
GO
USE DataEngineeringLab;
GO



SELECT * FROM DataEngineeringLab.dbo.Fact_Sales;
```

Output (result of SELECT * FROM DataEngineeringLab.dbo.Fact_Sales):

	transaction_id	category	price	quantity	transaction_date	total_value
1	101	ELECTRONICS	250.5	2	2023-01-15 00:00:00.000	501
2	103	FURNITURE	120	4	2023-01-17 00:00:00.000	480
3	104	ELECTRONICS	899.99	1	2023-02-10 00:00:00.000	899.99
4	105	BOOKS	45	3	2023-02-12 00:00:00.000	135
5	106	SPORTS	55	2	2023-03-05 00:00:00.000	110
6	107	CLOTHING	40	1	2023-03-08 00:00:00.000	40

Practical No. – 8

Aim:

To implement PySpark DataFrame operations for reading and analyzing CSV data, including schema inspection, filtering, grouping, aggregation, duplicate removal, joining datasets, and calculating average sales by product category.

1. Create the Spark session

Code:

```
!pip install pyspark

from pyspark.sql import SparkSession
from pyspark.sql.functions import col, avg

# Initialize Spark Session
spark = SparkSession.builder \
    .appName("PySpark Operations Demo") \
    .getOrCreate()

print("Spark Session Created Successfully!")
```

Output:

```
Requirement already satisfied: pyspark in /usr/local/lib/python3.13/dist-packages (4.0.4)
Requirement already satisfied: py4j<0.10.9.10,>=0.10.9.7 in /usr/local/lib/python3.13/dist-packages (from
    pyspark) (0.10.9.9)
Spark Session Created Successfully!
```

2. Create the sample CSV files

Code:

```
import pandas as pd

sales_raw = {
    'transaction_id': [1, 2, 3, 4, 5, 5, 6, 7],
    'product_id': [101, 102, 101, 103, 102, 102, 104, 101],
    'amount': [150.0, 300.0, 150.0, 450.0, 300.0, 300.0, 100.0, 200.0],
    'region': ['North', 'South', 'North', 'East', 'South', 'South', 'West', 'East']
}
pd.DataFrame(sales_raw).to_csv('sales_data.csv', index=False)

products_raw = {
    'product_id': [101, 102, 103, 104],
    'product_name': ['Laptop', 'Smartphone', 'Tablet', 'Headphones'],
    'category': ['Electronics', 'Electronics', 'Electronics', 'Accessories']
}
pd.DataFrame(products_raw).to_csv('products_data.csv', index=False)

print("Mock CSV files 'sales_data.csv' and 'products_data.csv' successfully created!")
```

Output:

```
Mock CSV files 'sales_data.csv' and 'products_data.csv' successfully created!
```

3. Read the CSV and inspect the schema

Code:

```
sales_df = spark.read.csv('sales_data.csv', header=True, inferSchema=True)

sales_df.printSchema()
```

```
sales_df.show()
```

Output:

```
root
 |-- transaction_id: integer (nullable = true)
 |-- product_id: integer (nullable = true)
 |-- amount: double (nullable = true)
 |-- region: string (nullable = true)

+--------------+----------+------+------+
|transaction_id|product_id|amount|region|
+--------------+----------+------+------+
|             1|       101| 150.0| North|
|             2|       102| 300.0| South|
|             3|       101| 150.0| North|
|             4|       103| 450.0|  East|
|             5|       102| 300.0| South|
|             5|       102| 300.0| South|
|             6|       104| 100.0|  West|
|             7|       101| 200.0|  East|
+--------------+----------+------+------+
```

4. Remove duplicate records

Code:

```
deduplicated_sales_df = sales_df.dropDuplicates()

print("Original Count:", sales_df.count())
print("Deduplicated Count:", deduplicated_sales_df.count())
deduplicated_sales_df.show()
```

Output:

```
Original Count: 8
Deduplicated Count: 7
+--------------+----------+------+------+
|transaction_id|product_id|amount|region|
+--------------+----------+------+------+
|             4|       103| 450.0|  East|
|             6|       104| 100.0|  West|
|             1|       101| 150.0| North|
|             7|       101| 200.0|  East|
|             3|       101| 150.0| North|
|             2|       102| 300.0| South|
|             5|       102| 300.0| South|
+--------------+----------+------+------+
```

5. Filter, group and aggregate

Code:

```
filtered_agg_df = deduplicated_sales_df \
```

```
    .filter(col("amount") > 100) \
    .groupBy("region") \
    .sum("amount") \
    .withColumnRenamed("sum(amount)", "total_sales")

filtered_agg_df.show()
```

Output:

```
+------+-----------+
|region|total_sales|
+------+-----------+
| South|      600.0|
|  East|      650.0|
| North|      300.0|
+------+-----------+
```

6. Join with the products dataset

Code:

```
products_df = spark.read.csv('products_data.csv', header=True, inferSchema=True)

joined_df = deduplicated_sales_df.join(products_df, on="product_id", how="inner")

joined_df.show()
```

Output:

```
+----------+--------------+------+------+------------+-----------+
|product_id|transaction_id|amount|region|product_name|   category|
+----------+--------------+------+------+------------+-----------+
|       103|             4| 450.0|  East|      Tablet|Electronics|
|       104|             6| 100.0|  West|  Headphones|Accessories|
|       101|             1| 150.0| North|      Laptop|Electronics|
|       101|             7| 200.0|  East|      Laptop|Electronics|
|       101|             3| 150.0| North|      Laptop|Electronics|
|       102|             2| 300.0| South|  Smartphone|Electronics|
|       102|             5| 300.0| South|  Smartphone|Electronics|
+----------+--------------+------+------+------------+-----------+
```

7. Average sales by product category

Code:

```
avg_sales_by_category = joined_df \
    .groupBy("category") \
    .agg(avg("amount").alias("average_sales"))

avg_sales_by_category.show()
```

Output:

```
+-----------+-----------------+
|   category|    average_sales|
+-----------+-----------------+
|Electronics|258.3333333333333|
|Accessories|            100.0|
+-----------+-----------------+
```

Practical No. – 9

Aim:

To design and implement an end-to-end data pipeline that extracts e-commerce data from CSV files, cleans and transforms the data using Python/Pandas, loads it into a SQL database, generates a report, and handles both historical and newly arriving data.

Input files (ecommerce_data):

customers.csv

```
customer_id,name,email,signup_date
1,alice smith,alice@email.com,2023-01-10
2,bob jones,,2023-01-15
3,charlie brown,charlie@email.com,2023-02-01
```

products.csv

```
product_id,product_name,category,price
101,Laptop,Electronics,1200.00
102,Wireless Mouse,Electronics,25.50
103,Desk Chair,Furniture,-50.00
```

orders.csv

```
order_id,customer_id,product_id,quantity,order_date
1001,1,101,1,2023-10-01
1002,2,102,2,2023-10-02
1003,3,101,1,2023-10-05
1004,1,102,1,2023-10-06
```

Code:

```
import pandas as pd
import os
from sqlalchemy import create_engine
from sqlalchemy.engine import URL

def setup_dummy_data():
    os.makedirs('ecommerce_data', exist_ok=True)

    if os.path.exists('ecommerce_data/orders.csv'):
        print("  -> Data files already exist. Skipping dummy generation so your new data is
safe.")
        return

    with open('ecommerce_data/customers.csv', 'w') as f:
        f.write("customer_id,name,email,signup_date\n")
        f.write("1,alice smith,alice@email.com,2023-01-10\n")
        f.write("2,bob jones,,2023-01-15\n")
        f.write("3,charlie brown,charlie@email.com,2023-02-01\n")

    with open('ecommerce_data/products.csv', 'w') as f:
        f.write("product_id,product_name,category,price\n")
        f.write("101,Laptop,Electronics,1200.00\n")
        f.write("102,Wireless Mouse,Electronics,25.50\n")
        f.write("103,Desk Chair,Furniture,-50.00\n")

    with open('ecommerce_data/orders.csv', 'w') as f:
        f.write("order_id,customer_id,product_id,quantity,order_date\n")
        f.write("1001,1,101,1,2023-10-01\n")
        f.write("1002,2,102,2,2023-10-02\n")
        f.write("1003,3,101,1,2023-10-05\n")
        f.write("1004,1,102,1,2023-10-06\n")

def load_incrementally(df, table_name, primary_key, engine):
    try:
```

```
        query = f"SELECT {primary_key} FROM {table_name}"
        existing_keys = pd.read_sql(query, engine)[primary_key].tolist()

        df_new = df[~df[primary_key].isin(existing_keys)]
    except Exception:
        df_new = df

    if not df_new.empty:
        df_new.to_sql(table_name, engine, if_exists='append', index=False)
        print(f"  -> SUCCESS: Inserted {len(df_new)} new records into '{table_name}'.")
    else:
        print(f"  -> SUCCESS: '{table_name}' is already up to date. No new records inserted.")

def run_pipeline():
    print("--- Starting Practical 9 E-Commerce ETL Pipeline ---\n")

    print("[0/5] Checking for data files...")
    setup_dummy_data()

    print("\n[1/5] EXTRACT: Reading Data from CSVs...")
    print("\n[1/5] EXTRACT: Reading Data from CSVs...")
    df_cust = pd.read_csv('ecommerce_data/customers.csv')
    df_prod = pd.read_csv('ecommerce_data/products.csv')
    df_ord = pd.read_csv('ecommerce_data/orders.csv')

    print("\n[2/5] TRANSFORM: Cleaning and Formatting Data...")
    df_cust['name'] = df_cust['name'].str.title()
    df_cust['email'] = df_cust['email'].fillna('Unknown')
    df_cust['signup_date'] = pd.to_datetime(df_cust['signup_date'])

    df_prod = df_prod[df_prod['price'] > 0]

    df_ord['order_date'] = pd.to_datetime(df_ord['order_date'])

    print("\n[3/5] LOAD: Connecting to MS SQL Server...")
    server_name = r'DESKTOP-P5RT5RO'
    database_name = 'ECommerceDB'

    connection_url = URL.create(
        "mssql+pyodbc",
        host=server_name,
        database=database_name,
        query={"driver": "ODBC Driver 17 for SQL Server", "Trusted_Connection": "yes"}
    )
    engine = create_engine(connection_url, fast_executemany=True)

    print("\n[4/5] LOAD: Executing Incremental Database Inserts...")
    load_incrementally(df_cust, 'dim_customers', 'customer_id', engine)
    load_incrementally(df_prod, 'dim_products', 'product_id', engine)
    load_incrementally(df_ord, 'fact_orders', 'order_id', engine)

    print("\n[5/5] REPORT: Generating Automated Analytics Report...")
    report_query = """
        SELECT
            p.product_name,
            SUM(o.quantity) as total_units_sold,
            SUM(o.quantity * p.price) as total_revenue
        FROM fact_orders o
        JOIN dim_products p ON o.product_id = p.product_id
        GROUP BY p.product_name
        ORDER BY total_revenue DESC;
    """

    df_report = pd.read_sql(report_query, engine)

    print("\n================ BUSINESS REPORT ================")
    print(df_report.to_string(index=False))
    print("=================================================\n")

    df_report.to_csv('ecommerce_data/final_sales_report.csv', index=False)
    print("Pipeline Complete! Report saved to 'ecommerce_data/final_sales_report.csv'.")
```

```
if __name__ == "__main__":
    run_pipeline()
```

Output:

```
--- Starting Practical 9 E-Commerce ETL Pipeline ---

[0/5] Checking for data files...

[1/5] EXTRACT: Reading Data from CSVs...

[1/5] EXTRACT: Reading Data from CSVs...

[2/5] TRANSFORM: Cleaning and Formatting Data...

[3/5] LOAD: Connecting to MS SQL Server...

[4/5] LOAD: Executing Incremental Database Inserts...
  -> SUCCESS: Inserted 3 new records into 'dim_customers'.
  -> SUCCESS: Inserted 2 new records into 'dim_products'.
  -> SUCCESS: Inserted 4 new records into 'fact_orders'.

[5/5] REPORT: Generating Automated Analytics Report...

================ BUSINESS REPORT ================
  product_name  total_units_sold  total_revenue
        Laptop                 2         2400.0
Wireless Mouse                 3           76.5
=================================================

Pipeline Complete! Report saved to 'ecommerce_data/final_sales_report.csv'.
```

Generated report file (final_sales_report.csv):

```
product_name,total_units_sold,total_revenue
Laptop,2,2400.0
Wireless Mouse,3,76.5
```

SQL Script (SQL Server):

```
CREATE DATABASE ECommerceDB;
GO

SELECT * FROM dim_customers;
SELECT * FROM dim_products;
SELECT * FROM fact_orders;
```

Output (SELECT * FROM dim_customers):

	customer_id	name	email	signup_date
1	1	Alice Smith	alice@email.com	2023-01-10 00:00:00.000
2	2	Bob Jones	Unknown	2023-01-15 00:00:00.000
3	3	Charlie Brown	charlie@email.com	2023-02-01 00:00:00.000

Output (SELECT * FROM dim_products):

	product_id	product_name	category	price
1	101	Laptop	Electronics	1200.00
2	102	Wireless Mouse	Electronics	25.50

Output (SELECT * FROM fact_orders):

	order_id	customer_id	product_id	quantity	order_date
1	1001	1	101	1	2023-10-01 00:00:00.000
2	1002	2	102	2	2023-10-02 00:00:00.000
3	1003	3	101	1	2023-10-05 00:00:00.000
4	1004	1	102	1	2023-10-06 00:00:00.000

Mini Project – Retail Data Warehouse ETL Pipeline

Aim:

To design and implement a retail data warehouse ETL pipeline in Python that ingests store, product and transaction data from CSV and JSON files, applies data-quality rules, shapes the data into a star schema, loads it incrementally into SQL Server, and generates a revenue report.

Input files (source_data):

stores.csv

```
store_id,location,region
1,Mumbai,West
2,Pune,West
3,Delhi,North
```

products.json

```
[
        {"product_id": 101, "name": "Apples", "category": "Groceries", "unit_price": 120},
        {"product_id": 102, "name": "Milk", "category": "Dairy", "unit_price": 65},
        {"product_id": 103, "name": "Bread", "category": "Bakery", "unit_price": -20}
    ]
```

transactions.csv

```
transaction_id,store_id,product_id,quantity,sale_date
1001,1,101,5,2026-10-01
1002,1,102,2,2026-10-01
1003,2,101,10,2026-10-02
1004,,103,1,2026-10-02
```

Code:

```
import pandas as pd
import os
from sqlalchemy import create_engine
from sqlalchemy.engine import URL



def generate_source_data():
    os.makedirs('source_data', exist_ok=True)

    if os.path.exists('source_data/transactions.csv'):
        print("  -> Source files already exist. Skipping generation.")
        return

    with open('source_data/stores.csv', 'w') as f:
        f.write("store_id,location,region\n1,Mumbai,West\n2,Pune,West\n3,Delhi,North\n")

    products_json = """[
        {"product_id": 101, "name": "Apples", "category": "Groceries", "unit_price": 120},
        {"product_id": 102, "name": "Milk", "category": "Dairy", "unit_price": 65},
        {"product_id": 103, "name": "Bread", "category": "Bakery", "unit_price": -20}
    ]"""
    with open('source_data/products.json', 'w') as f:
        f.write(products_json)

    with open('source_data/transactions.csv', 'w') as f:
        f.write("transaction_id,store_id,product_id,quantity,sale_date\n")
        f.write("1001,1,101,5,2026-10-01\n")
        f.write("1002,1,102,2,2026-10-01\n")
        f.write("1003,2,101,10,2026-10-02\n")
        f.write("1004,,103,1,2026-10-02\n")

def load_to_dw(df, table_name, pk_column, engine):
    try:
```

```
        existing_keys = pd.read_sql(f"SELECT {pk_column} FROM {table_name}",
engine)[pk_column].tolist()
        df_new = df[~df[pk_column].isin(existing_keys)]
    except Exception:
        df_new = df

    if not df_new.empty:
        df_new.to_sql(table_name, engine, if_exists='append', index=False)
        print(f"  -> SUCCESS: Loaded {len(df_new)} new records into '{table_name}'.")
    else:
        print(f"  -> SUCCESS: '{table_name}' is up to date.")

def execute_pipeline():
    print("=== Starting Retail Data Warehouse ETL Pipeline ===\n")

    print("[1/5] DATA INGESTION: Sourcing raw data...")
    generate_source_data()

    df_stores = pd.read_csv('source_data/stores.csv')
    df_products = pd.read_json('source_data/products.json')
    df_sales = pd.read_csv('source_data/transactions.csv')

    print("\n[2/5] DATA CLEANING: Applying quality rules...")
    invalid_price_count = len(df_products[df_products['unit_price'] <= 0])
    df_products = df_products[df_products['unit_price'] > 0]

    missing_fk_count = df_sales['store_id'].isna().sum()
    df_sales = df_sales.dropna(subset=['store_id'])
    print(f"  -> Removed {invalid_price_count} invalid products and {missing_fk_count}
orphaned sales records.")

    print("\n[3/5] TRANSFORMATION: Shaping data for Star Schema...")
    df_sales['sale_date'] = pd.to_datetime(df_sales['sale_date'])

    df_sales_enriched = df_sales.merge(df_products[['product_id', 'unit_price']],
on='product_id', how='left')
    df_sales['total_amount'] = df_sales_enriched['quantity'] * df_sales_enriched['unit_price']

    df_sales['store_id'] = df_sales['store_id'].astype(int)

    print("\n[4/5] STORAGE: Loading into SQL Server Star Schema...")
    server_name = r'DESKTOP-P5RT5RO'
    connection_url = URL.create(
        "mssql+pyodbc",
        host=server_name,
        database='RetailDW',
        query={"driver": "ODBC Driver 17 for SQL Server", "Trusted_Connection": "yes"}
    )
    engine = create_engine(connection_url, fast_executemany=True)

    load_to_dw(df_stores, 'Dim_Store', 'store_id', engine)
    load_to_dw(df_products, 'Dim_Product', 'product_id', engine)
    load_to_dw(df_sales, 'Fact_Sales', 'transaction_id', engine)

    print("\n[5/5] REPORTING: Generating Analytical Output...")
    report_query = """
        SELECT
            s.location,
            p.category,
            SUM(f.quantity) as total_units_sold,
            SUM(f.total_amount) as total_revenue
        FROM Fact_Sales f
        JOIN Dim_Store s ON f.store_id = s.store_id
        JOIN Dim_Product p ON f.product_id = p.product_id
        GROUP BY s.location, p.category
        ORDER BY total_revenue DESC;
    """
    df_report = pd.read_sql(report_query, engine)
    print("\n=== EXECUTIVE REVENUE REPORT ===")
    print(df_report.to_string(index=False))
    print("================================\n")
```

```
    df_report.to_csv('source_data/warehouse_revenue_report.csv', index=False)
    print("Pipeline Execution Completed Successfully.")

if __name__ == "__main__":
    execute_pipeline()
```

Output:

```
=== Starting Retail Data Warehouse ETL Pipeline ===

[1/5] DATA INGESTION: Sourcing raw data...
  -> Source files already exist. Skipping generation.

[2/5] DATA CLEANING: Applying quality rules...
  -> Removed 1 invalid products and 1 orphaned sales records.

[3/5] TRANSFORMATION: Shaping data for Star Schema...

[4/5] STORAGE: Loading into SQL Server Star Schema...
  -> SUCCESS: Loaded 3 new records into 'Dim_Store'.
  -> SUCCESS: Loaded 2 new records into 'Dim_Product'.
  -> SUCCESS: Loaded 3 new records into 'Fact_Sales'.

[5/5] REPORTING: Generating Analytical Output...

=== EXECUTIVE REVENUE REPORT ===
location  category  total_units_sold  total_revenue
    Pune Groceries                10           1200
  Mumbai Groceries                 5            600
  Mumbai     Dairy                 2            130
================================

Pipeline Execution Completed Successfully.
```

Generated report file (warehouse_revenue_report.csv):

```
location,category,total_units_sold,total_revenue
Pune,Groceries,10,1200
Mumbai,Groceries,5,600
Mumbai,Dairy,2,130
```

SQL Script (SQL Server):

```
CREATE DATABASE RetailDW;
GO

SELECT * FROM Fact_Sales;
```

Output (SELECT * FROM Fact_Sales):

	transaction_id	store_id	product_id	quantity	sale_date	total_amount
1	1001	1	101	5	2026-10-01 00:00:00.000	600
2	1002	1	102	2	2026-10-01 00:00:00.000	130
3	1003	2	101	10	2026-10-02 00:00:00.000	1200